Alexandra Chase and Elios Hoxholli
amchas26@g.holycross.edu, ehoxho27@g.holycross.edu
CSCI 347 Artificial Intelligence
27 March 2026
Project 2 Write Up

Description of Problem:

Star Battle is a puzzle. This game is played on an n * n grid divided into n number of distinct regions. The goal is to place exactly k (given) stars in the grid such that the following constraints are satisfied simultaneously:

1. Every row contains exactly k stars.
2. Every column contains exactly k stars.
3. Every region contains exactly k stars.
4. No two stars may be adjacent to one another, including diagonally.

For our purposes, we consider puzzles where k is equal to 1, 2, or 3. Violating any one of the above constraints makes the placement invalid. In some instances, a puzzle will have no valid solution at all. A placement that satisfies the row and column constraints may violate a region constraint, and satisfying all region constraints may introduce adjacency violations. We address this by encoding the entire problem as a boolean satisfiability instance (using PySAT) and delegating a BFS search for a valid assignment to a SAT solver.

Formal Description of Encoding via SAT:

We use a function f(r,c) = (r*n) + c + 1 to map each $X_{r,c}$ as an integer. If the function returns an *+i,* then $X_{r,c}$= True (a star is in the cell). If the function returns *-i*, then $X_{r,c}$= False (no star in the cell). The formal encoding via SAT in our program is derived from:

1. All CNF clauses produced by the sequential counter encodings which follow/satisfy : $\sum(c=0)$ Xr,c = k and $\sum(r=0)$ Xr,c = k (for every row and column) and $\sum(r,c \in Ri)$ Xr,c = k, where set R contains regions: R={R1, R2, R3, …, Rm }.
2. The CNF clause for every cell to the adjacency constraint: (¬XAr,c ∨ ¬XBr-1,c ¬XBr-1,c-1 ∨ ¬XBr,c+1 ¬XBr+1,c+1 ∨ ¬XBr+1,c ¬XBr+1,c-1 ∨ ¬XBr,c-1 ¬XBr-1,c-1) where:
   - XAr,c→ current cell
   - XBr,c→ neighbour cell(s)
3. The constraints for exactly k stars in each row, column, and region all follow the same format as our last homework:
   - Columns: (Xr,1 ∨ Xr,2 ∨ Xr,3 v [...] v Xr,n) ∧ (¬Xr,1 ∨ ¬Xr,2 ∨ ¬Xr,3 v [...] v ¬Xr,n)
   - Rows: (¬X1,c ∨ ¬X2,c ∨ ¬X3,c v [...] v ¬Xn,c) ∧ (¬X1,c ∨ ¬X2,c ∨ ¬X3,c v [...] v ¬Xn,c)

- Regions: (¬Xr0,c0 ∨ ¬Xr1,c1 ∨ ¬Xr2,2c v [...] v ¬XNr,Nc) ∧ (¬X0r,0c ∨ ¬Xr1,1c ∨ ¬X2r,2c v [...] v ¬XNr,Nc)

Discussion of Our Implementation:

1. **Parsing**
   First, a puzzle is read from a plain text file. The first line of that file gives k's value on its own line. The remaining lines form the raw character grid, which has dimensions (2n+1) by (2n+1), which includes characters depicting walls ('|' , '-') and open spaces (' '). For example, a puzzle cell at position (r, c) in playable coordinates maps to the raw grid position (2r+1, 2c+1). This raw char grid is then stripped of new line characters and held as "grid" in a list of rows (lists). N is derived by removing the "barrier" characters and only accounts for playable puzzle cells. It then returns n, l, and the grid.

2. **Region Extraction**
   Regions are identified using BFS over the raw coordinate grid. We iterate over all odd-indexed row, col positions, which correspond to playable puzzle cells. For each *unvisited cell* we start a BFS, expanding its neighbors only when no wall character blocks the path. Each connected cell found this way is collectively one region. The BFS returns a complete list of regions, containing separate lists of cells in raw coordinates. These are later converted to playable coordinates when building the SAT encoding.

3. **SAT Encoding**
   A CNF formula is a conjunction of clauses, where each clause is a disjunction of literals. The encode function builds this formula incrementally, as each loop contributes its own independent batch of clauses to the same growing CNF object. We use PySAT's CNF class to build these clauses and CardEnc.equals to encode the cardinality constraints.

   - **Exactly k stars in each row:** A loop iterates over each row r from 0 to n-1. For each row it collects the SAT IDs of all cells in that row and passes them to CardEnc.equals with bound k. CardEnc.equals compiles this into a set of satisfiable CNF clauses. Those clauses are immediately added to the CNF formula.
   - **Exactly k stars in each column:** Does the same thing but fixes c and varies r. Even though the loop structure looks identical to the row loop, it is a separate loop because it groups cells differently. These are different constraints over different groups of variables and must be encoded independently.
   - **Exactly k stars in each region:** Iterates over the list of regions produced by BFS. Each region is an irregular group of cells of arbitrary shape and size. Because

regions come back from BFS in raw coordinates, each (r, c) is first converted to playable coordinates with (r-1)//2, (c-1)//2.

- **No two stars in adjacent cells:** Rather than encoding a count constraint over a group, it encodes a pairwise prohibition. For every cell (r, c) it checks all 8 neighbors. For each valid neighboring cell (r2, c2) it appends the clause [-var(r,c,n), -var(r2,c2,n)] directly to the formula. This clause is already in CNF form as it is a disjunction of two negated literals. The check (r,c) < (r2,c2) ensures each pair is only added once. After this loop, every possible adjacent pair of cells has a clause forbidding them from both being stars.

The conjunction of all those clauses is what forces every row, column, region, and adjacency constraint to hold at the same time in any satisfying assignment.

4. **Solving**

Once the CNF formula is built, it is handed to Glucose3 which returns either a satisfying assignment or reports that none exists. The model is a list of signed integers where a positive value means that variable was assigned true and a negative value means false. We convert it to a set so that the membership check is fast. Any cell variable that appears true (positive) in the model corresponds to a star placement, and we collect those (r, c) pairs. To print the solution we make a copy of the raw character grid and write '*' into each star's position. Since stars are stored in playable coordinates (r, c), but the raw grid uses the (2r+1, 2c+1) mapping, we have to convert back before writing.